Universidad Politécnica Metropolitana del Estado de Hidalgo

Maestría en Inteligencia Artificial

Funciones de Aptitud y Evaluación
Ajuste de Parámetros y Convergencia

Conceptos, implementación en Python y ejercicios prácticos

Materia: Algoritmos Evolutivos
Alumna: Jessica Melani Romero Lora
Profesor: Dr. Jaime Aguilar Ortiz

Octubre 2026

1. Definiciones prácticas

- **Función de aptitud (fitness):** Función que asigna un valor numérico a cada individuo, midiendo qué tan buena es la solución que representa. Es el motor de la selección natural dentro del AG.
- **Evaluación:** Proceso de aplicar la función de fitness a cada individuo de la población para obtener su valor de aptitud.
- **Paisaje de fitness:** Representación conceptual del espacio de búsqueda donde cada punto tiene un valor de fitness asociado. Puede tener múltiples picos (óptimos locales) y valles.
- **Convergencia:** Tendencia de la población a concentrarse en una región del espacio de búsqueda. Es deseable cuando se dirige al óptimo global, y problemática cuando es prematura.
- **Convergencia prematura:** Situación en la que la población pierde diversidad demasiado pronto y queda atrapada en un óptimo local, sin explorar otras regiones prometedoras.
- **Presión selectiva:** Grado en que los individuos más aptos dominan la reproducción. Demasiada presión causa convergencia prematura; muy poca ralentiza la búsqueda.

2. Funciones de aptitud y evaluación

2.1 El proceso de evaluación

La evaluación transforma un cromosoma en un valor numérico que el AG pueda comparar. El proceso sigue estos pasos:

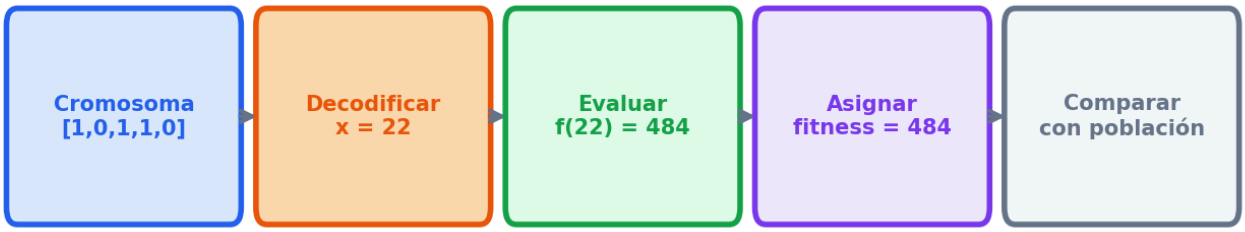


Figura 1. Pipeline de evaluación: del cromosoma al fitness.

2.2 Tipos de funciones de fitness

La función de fitness se diseña según el problema. En maximización directa, el fitness es el valor de la función objetivo. En minimización, se invierte (fitness = 1/f(x) o fitness = C − f(x)) para que valores menores produzcan mayor aptitud. Cuando existen restricciones, se aplican penalizaciones que reducen el fitness de las soluciones infactibles.

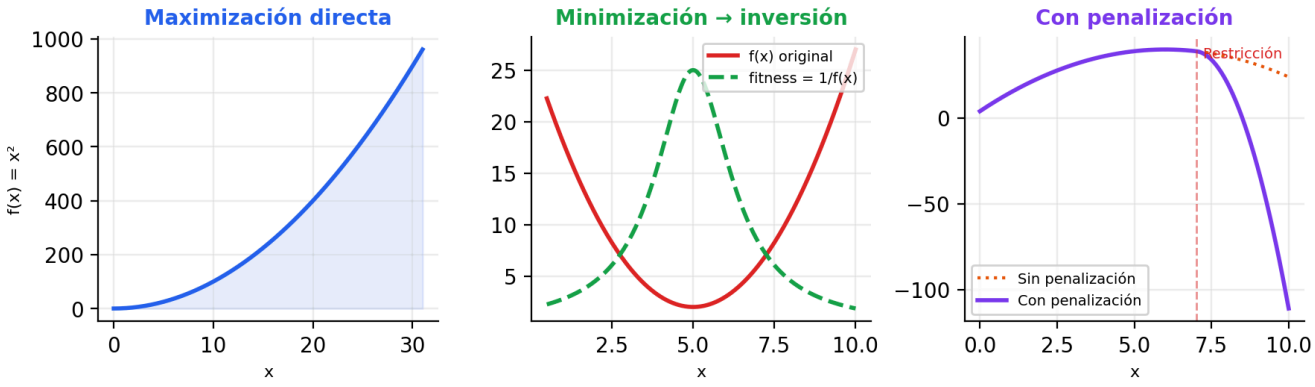


Figura 2. Tres estrategias: maximización directa, inversión para minimizar y penalización por restricciones.

2.3 Escalamiento del fitness

Cuando los valores de fitness están muy concentrados, las diferencias entre individuos son pequeñas y la selección pierde eficacia. El escalamiento lineal (f' = a·f + b) o el escalamiento por rango redistribuyen los valores para controlar la presión selectiva.

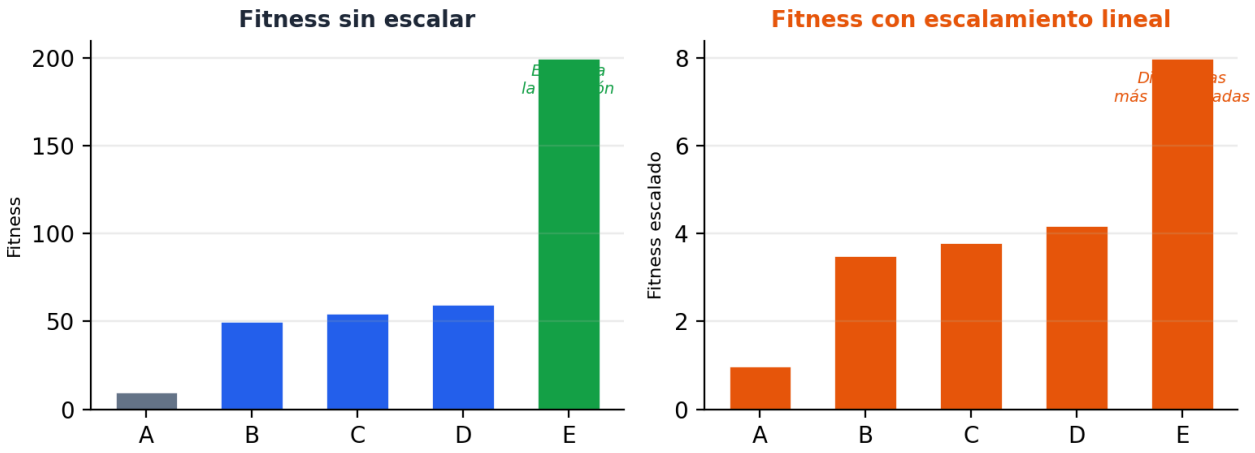


Figura 3. Sin escalar, el individuo E domina. Con escalamiento lineal, las diferencias se equilibran.

Técnica	Fórmula	Cuándo usarla
Lineal	$f' = a \cdot f + b$	Fitness con poca varianza
Sigma	$f' = \max(f - (\mu - c \cdot \sigma), 0)$	Distribución gaussiana del fitness
Por rango	$f' = N - \text{rango}(i)$	Evitar que outliers dominen
Boltzmann	$f' = e^{f/T}$	Control adaptativo (T baja con el tiempo)

Tabla 1. Técnicas de escalamiento de fitness.

3. Ajuste de parámetros

El rendimiento de un AG depende de cuatro parámetros principales: el tamaño de la población (N), la probabilidad de cruzamiento (Pc), la probabilidad de mutación (Pm) y el número de generaciones. Un mal ajuste puede provocar convergencia prematura o una exploración excesivamente lenta.

3.1 Tamaño de la población

Poblaciones pequeñas convergen rápido pero pierden diversidad; poblaciones grandes exploran mejor pero consumen más recursos. Un rango típico es N = 20–200, aunque depende del problema.

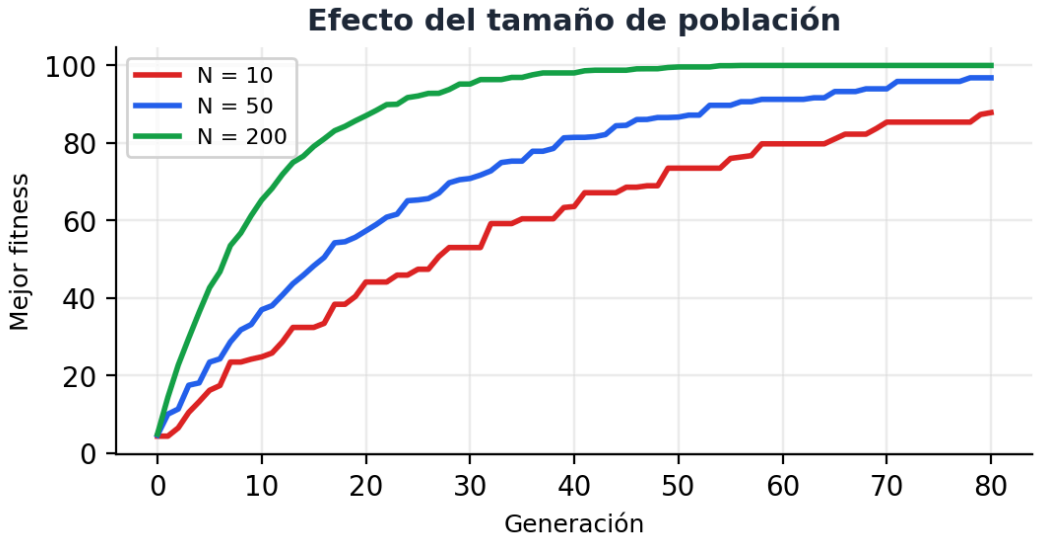


Figura 4. Con N = 10 la convergencia es ruidosa e inestable; N = 200 es más suave y confiable.

3.2 Probabilidades de cruzamiento y mutación

Pc controla cuántas parejas intercambian material (típico: 0.6–0.9). Pm controla la tasa de alteración aleatoria (típico: 0.001–0.05). Valores altos de Pm destruyen buenas soluciones; valores bajos reducen la exploración.

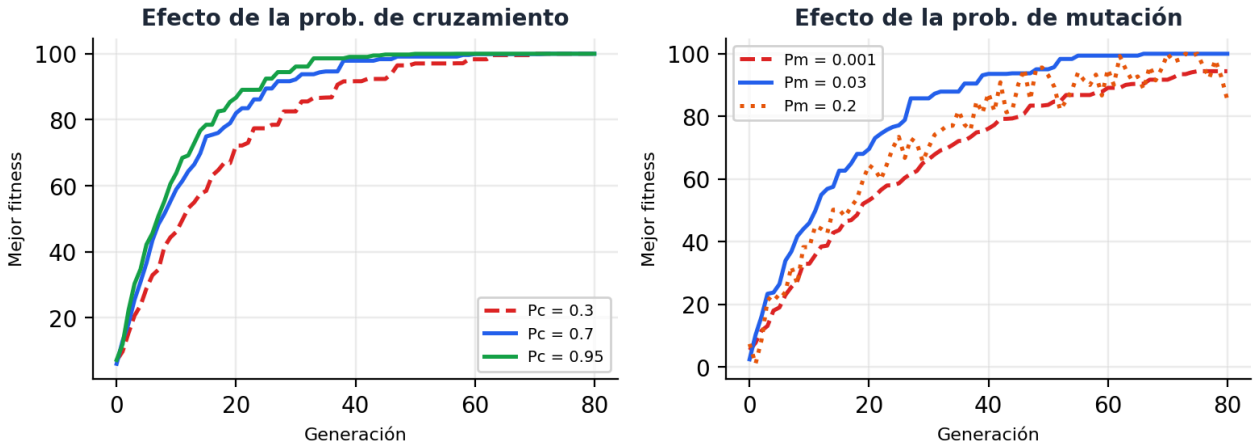


Figura 5. Izquierda: mayor Pc acelera la convergencia. Derecha: Pm = 0.03 es ideal; Pm = 0.2 genera inestabilidad.

Parámetro	Rango típico	Efecto si es muy bajo	Efecto si es muy alto
N (población)	20 – 200	Poca diversidad	Lento, costoso
Pc (cruzamiento)	0.6 – 0.9	Poca recombinación	Destruye esquemas buenos
Pm (mutación)	0.001 – 0.05	Convergencia prematura	Búsqueda aleatoria
Generaciones	50 – 500	No converge	Costo computacional alto

Tabla 2. Parámetros típicos y sus efectos.

4. Convergencia

La convergencia ocurre cuando la población deja de mejorar significativamente. Se distingue la convergencia sana (el AG alcanza el óptimo global) de la convergencia prematura (se estanca en un óptimo local). La diversidad genética es un indicador clave: si cae a cero, el AG ya no puede explorar.

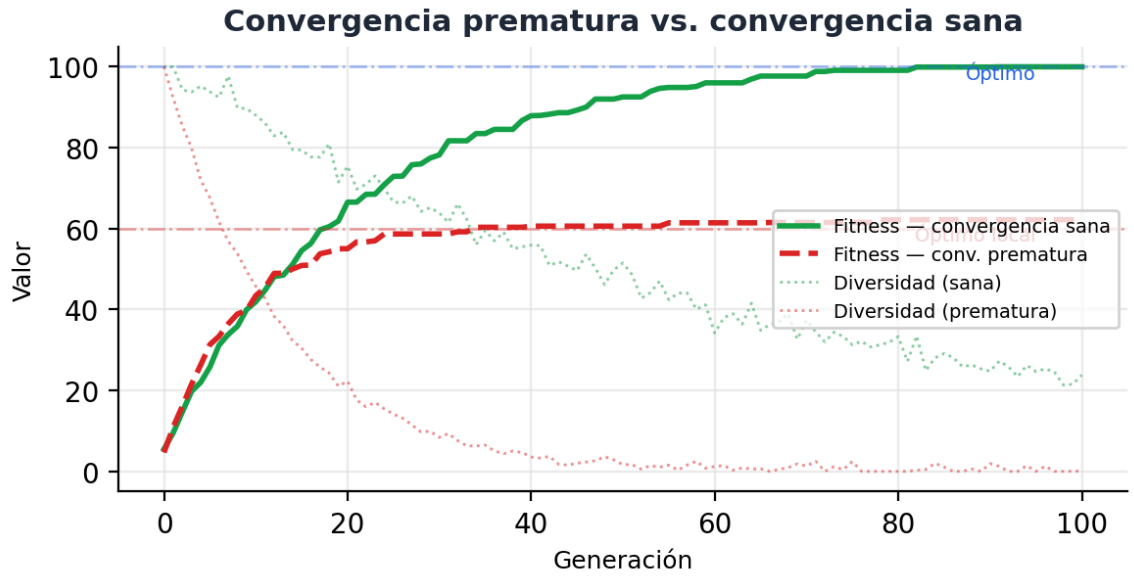


Figura 6. Convergencia sana (verde) alcanza el óptimo global; prematura (roja) se estanca en un óptimo local.

Criterios de parada comunes: (1) número máximo de generaciones, (2) fitness estancado durante k generaciones seguidas, (3) diversidad genética por debajo de un umbral, (4) se alcanza un fitness objetivo conocido.

5. Implementación en Python

El siguiente código implementa un AG con fitness escalado, registro de convergencia y gráfico automático:

```
import random, matplotlib.pyplot as plt

BITS, POP, GEN, PC, PM = 8, 30, 80, 0.8, 0.03

def decode(ind): return int(''.join(map(str,ind)),2)
def fitness(ind):                    # Maximizar f(x)=x·sin(x/10)
    x = decode(ind)
    return max(x * __import__('math').sin(x/10), 0.01)
```

```
def seleccion_torneo(pob, fits, k=3): # Torneo de tamaño k
    rivales = random.sample(list(zip(pob,fits)), k)
    return max(rivales, key=lambda t:t[1])[0][:]

def cruzamiento(p1, p2):
    if random.random()<PC:
        pt=random.randint(1,BITS-1)
        return p1[:pt]+p2[pt:], p2[:pt]+p1[pt:]
    return p1[:], p2[:]

def mutacion(ind):
    return [b if random.random()>PM else 1-b for b in ind]

pob = [[random.randint(0,1) for _ in range(BITS)] for _ in range(POP)]
historial = []
for g in range(GEN):
    fits = [fitness(i) for i in pob]
    historial.append(max(fits))       # Registrar mejor fitness
    nueva = []
    while len(nueva) < POP:
        p1 = seleccion_torneo(pob, fits)
        p2 = seleccion_torneo(pob, fits)
        h1, h2 = cruzamiento(p1, p2)
        nueva += [mutacion(h1), mutacion(h2)]
    pob = nueva[:POP]

plt.plot(historial); plt.xlabel('Generación')
plt.ylabel('Mejor fitness'); plt.title('Convergencia')
plt.savefig('convergencia.png'); plt.show()
```

Código 1. AG con selección por torneo, fitness no lineal y gráfico de convergencia.